Computer Organization and Assembly Language

Week 08 — Chapter 3 continued
Relative Addressing, Types of Jump and the Sorting Example

Dr. Maqsood M. Khan

Department of Computer Science / Software Engineering / Artificial Intelligence
FAST-NUCES Peshawar

Week 08

Outline

1. Quick Review
2. Why Are We Studying This?
3. Walk-through: Where Everything Sits in Memory
4. Relative Addressing
5. Types of Jump
6. The Sorting Example
7. Signed vs Unsigned: The Experiment

Quick Review: What Was Done Last Week?

- `CMP` subtracts the source from the destination, sets the flags and **discards the result**. It is meaningless without a conditional jump after it.
- Four flags — **ZF, CF, SF, OF** — express every relation between two operands.
- The **same bit pattern** has two meanings: `FFFE` is -2 signed and 65534 unsigned. The `CMP` does not know which you meant.
- **The jump declares your intent:** `JA`/`JB` $\rightarrow$ unsigned, `JG`/`JL` $\rightarrow$ signed.
  - Unsigned conditions use **ZF and CF**.
  - Signed conditions use **ZF, SF and OF**.
- `JCXZ` is the only jump that reads **no flag**.
- **Example 3.1** — `cmp bx, 20` replaced the CX counter entirely.
- `JMP` — the unconditional jump, used to **jump over data** placed at the start of a program.
- We noticed the jump operand was `F2`, not an address — and promised to explain it. That is where we start today.

Why Are We Studying This?

Two questions this week answers

1. **How does a jump actually find its target?** The opcode never contains the address we wrote.
2. **Can we now write a *real* program?** We have loops, comparisons and decisions — enough to build a genuine algorithm.

- **Relative addressing** explains why code can be loaded anywhere in memory and still run — the foundation of relocatable programs.
- **Jump types** explain a limit you *will* hit: a conditional jump cannot reach further than 128 bytes.
- **Sorting** is the first program where the logic is not trivial. It uses everything from the course so far: arrays, indexed addressing, comparison, conditional jumps, loops and a flag in memory.
- It also shows, concretely, what `signed` and `unsigned` mean in C++ — by sorting the same data two different ways.

The Program We Are Tracing

```
; a program to add ten numbers without a separate counter
[org 0x0100]
        jmp start ; unconditionally jump over data

num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total: dw 0

start: mov bx, 0 ; initialize array index to zero
        mov ax, 0 ; initialize sum to zero
l1: add ax, [num1+bx] ; add number to ax
        add bx, 2 ; advance bx to next index
        cmp bx, 20 ; are we beyond the last index?
        jne l1 ; if not, add next number
        mov [total], ax ; write back sum in memory
        mov ax, 0x4c00
        int 0x21
```

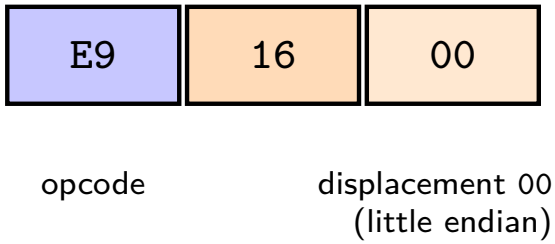
- Two jumps to follow: one **forward** over the data, one **backward** to repeat the loop.

Step 1 — The Memory Layout

Offset	Bytes	Instruction / data
0100	E9 16 00	jmp start (3 bytes)
0103	0A 00	num1 → 10
0105	14 00	→ 20
0107	1E 00	→ 30
0109	28 00	→ 40
010B	32 00	→ 50
010D	0A 00	→ 10
010F	14 00	→ 20
0111	1E 00	→ 30
0113	28 00	→ 40
0115	32 00	→ 50
0117	00 00	total
0119	BB 00 00	start: mov bx,0
011C	B8 00 00	mov ax,0
011F	03 87 03 01	l1: add ax,[num1+bx]
0123	81 C3 02 00	add bx,2
0127	81 FB 14 00	cmp bx,20
012B	75 F2	jne l1
012D	A3 17 01	mov [total],ax

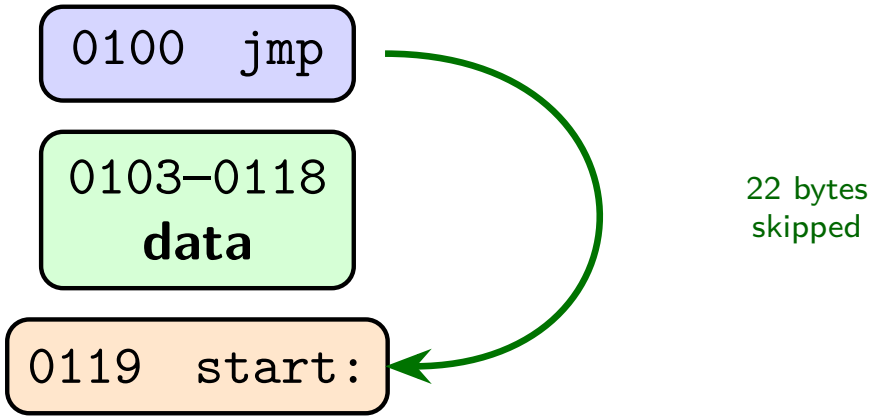
num1 = 0103 | total = 0117 | start = 0119 | l1 = 011F

Step 2 — The Forward Jump: `jmp start`

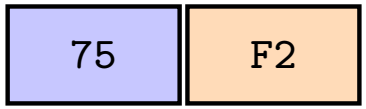


- The instruction is **3 bytes** long and sits at 0100, so by the time it executes IP has already advanced to 0103.

$$0103 + 0016 = 0119 = \texttt{start} \quad \checkmark$$



Step 3 — The Backward Jump: `jne l1`



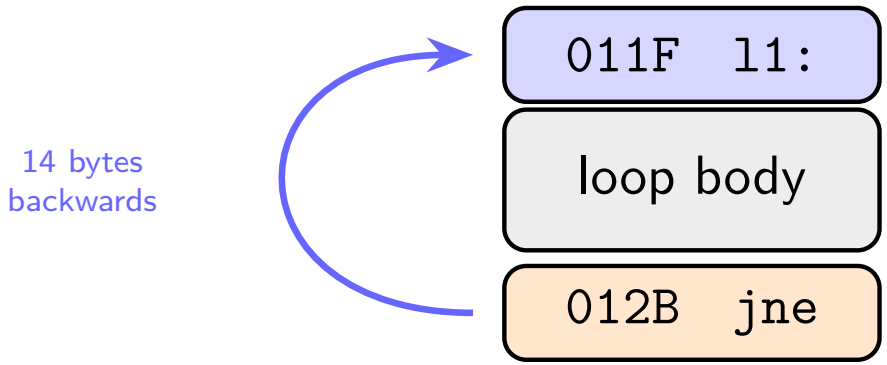
opcode JNZdisplacement

- F2 has its top bit set, so it is a **negative** signed byte:

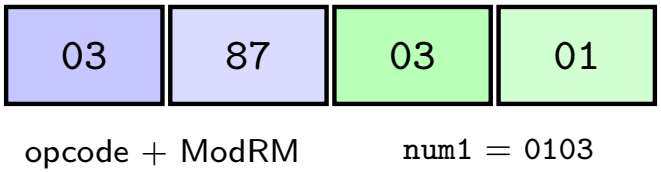
$$\texttt{F2}_{16} = 242, \qquad 242 - 256 = -\mathbf{14}$$

- The instruction is **2 bytes** at 012B, so IP has already advanced to 012D.

$$\texttt{012D} - \texttt{000E} = \texttt{011F} = \texttt{l1} \quad \checkmark$$



Step 4 — The Memory Access Changes Every Pass



$$\text{effective address} = 0103 + \text{BX}$$

Pass	BX	EA = 0103 + BX	value read	AX after
1	0000	0103	10	10
2	0002	0105	20	30
3	0004	0107	30	60
4	0006	0109	40	100
5	0008	010B	50	150
6	000A	010D	10	160
7	000C	010F	20	180
8	000E	0111	30	210
9	0010	0113	40	250
10	0012	0115	50	**300**

After pass 10, BX becomes 0014 (= 20), so `cmp bx,20` gives zero and `jne` is not taken. **300** is then written to 0117.

What the Three Operands Tell Us

Instruction	Operand	What it stores
`jmp start`	0016	a **distance** — 22 bytes forward
`jne l1`	F2	a **distance** — 14 bytes backward
`add ax,[num1+bx]`	0103	an **offset** inside the segment

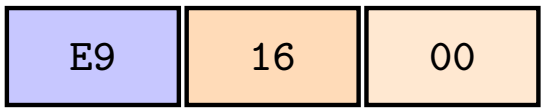
Why the program can be loaded anywhere

The two jumps store **gaps**, and a gap does not change when the whole program moves. The memory access stores an **offset**, and offsets are measured from the segment base — which moves with the program.

Nothing has to be rewritten at load time.

The Jump Operand Is Not an Address

- In the debugger our instruction is shown as `JMP 0119`, and location `0119` holds the first real instruction of our program.
- But look at the opcode bytes:



opcode

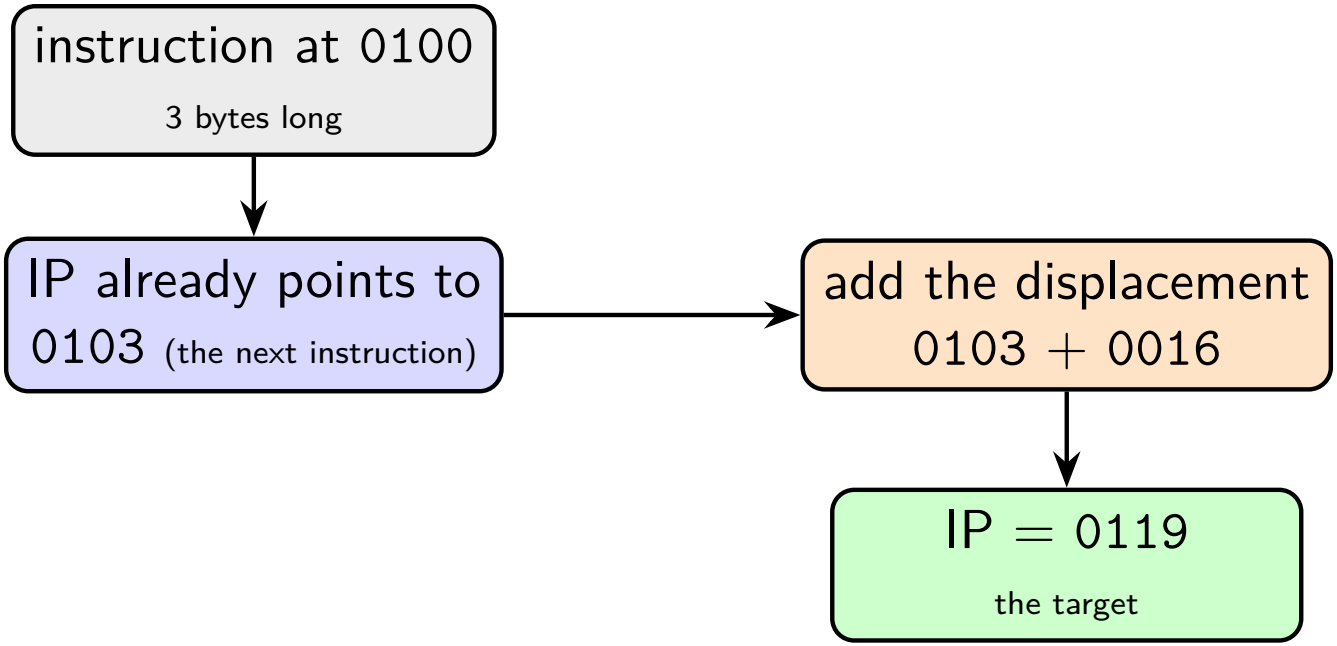
operand 0016
(little endian)

0119 does **not** appear
anywhere in the instruction!

What the operand really is

0016 is not a location. It says **how far forward to go from the current position of IP**. This is position-relative addressing, in contrast to **absolute** addressing.

How the Target Is Computed



$$0103 + 0016 = 0119 \quad \checkmark$$

- **Crucial detail:** the displacement is added to the address of the *next* instruction, not to the address of the jump itself. By the time the jump executes, IP has already advanced.
- **You never calculate this.** The assembler works out the displacement and the processor performs the addition — you simply write a **label**.

Relative vs Absolute — and Why It Matters

Relative

Stores a **distance**.

"Go 22 bytes forward."

Works no matter where the program is loaded.

Absolute

Stores an **address**.

"Go to `0119`."

Breaks if the program moves.

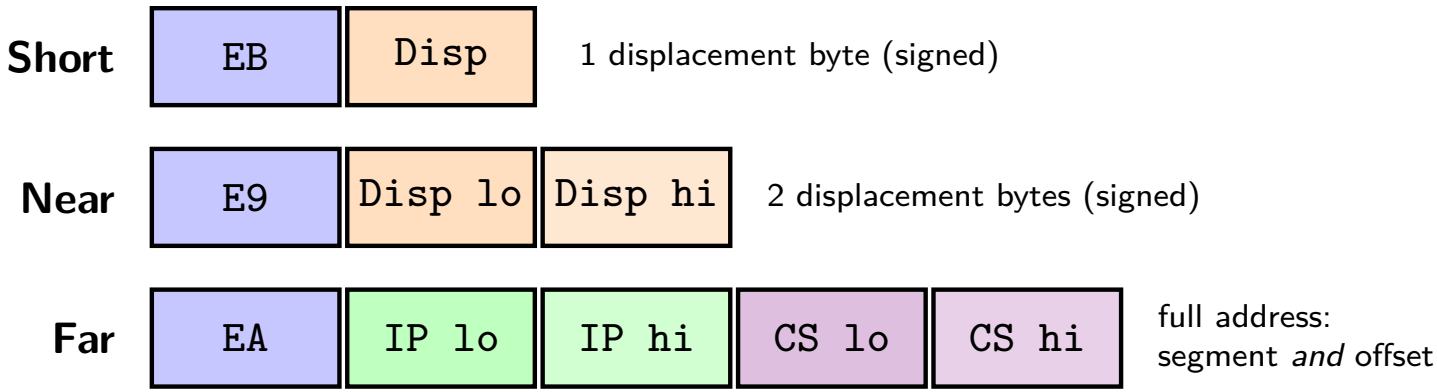
- A relative jump measures from wherever it happens to be, so the **whole block of code can be loaded at any offset** and every internal jump still works.
- This is why DOS can load your `.COM` file at `0100` today and somewhere else tomorrow without rewriting a single instruction.
- A **negative** displacement jumps backwards — which is exactly what a loop needs.

Three Types of Jump

Type	Size	Addressing	Range
Short	2 bytes	relative	-128 to $+127$ bytes — about 256 bytes in total
Near	3 bytes	relative	anywhere within the **current segment** (64 KB)
Far	5 bytes	**absolute**	anywhere in the whole **1 MB** of memory

- Bigger range costs more bytes — the classic trade-off.
- The assembler picks the smallest form that can reach the target, so you normally never think about it.

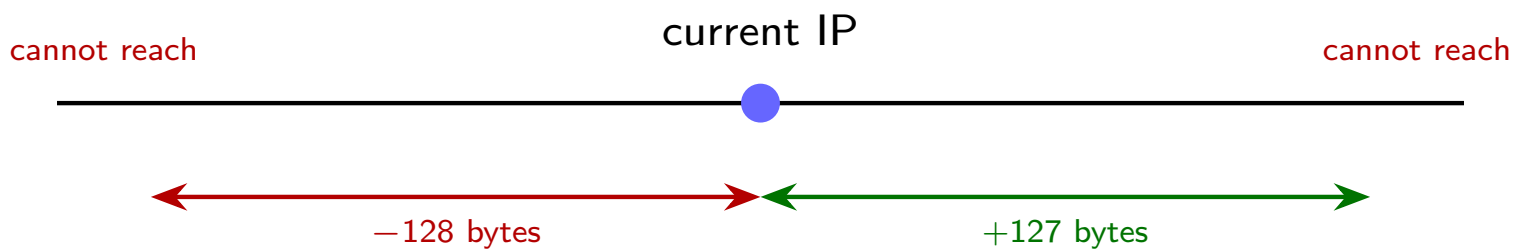
How Each One Is Encoded



- Short and near carry a **displacement** — a distance.
- Far carries an **address** — both halves of it.

Short Jump

- The displacement is stored in a **single byte**, e.g. `75 F2` — opcode `75`, operand `F2`.
- `F2` is added to IP **as a signed byte**: if negative, the jump goes backwards; if positive, forwards.

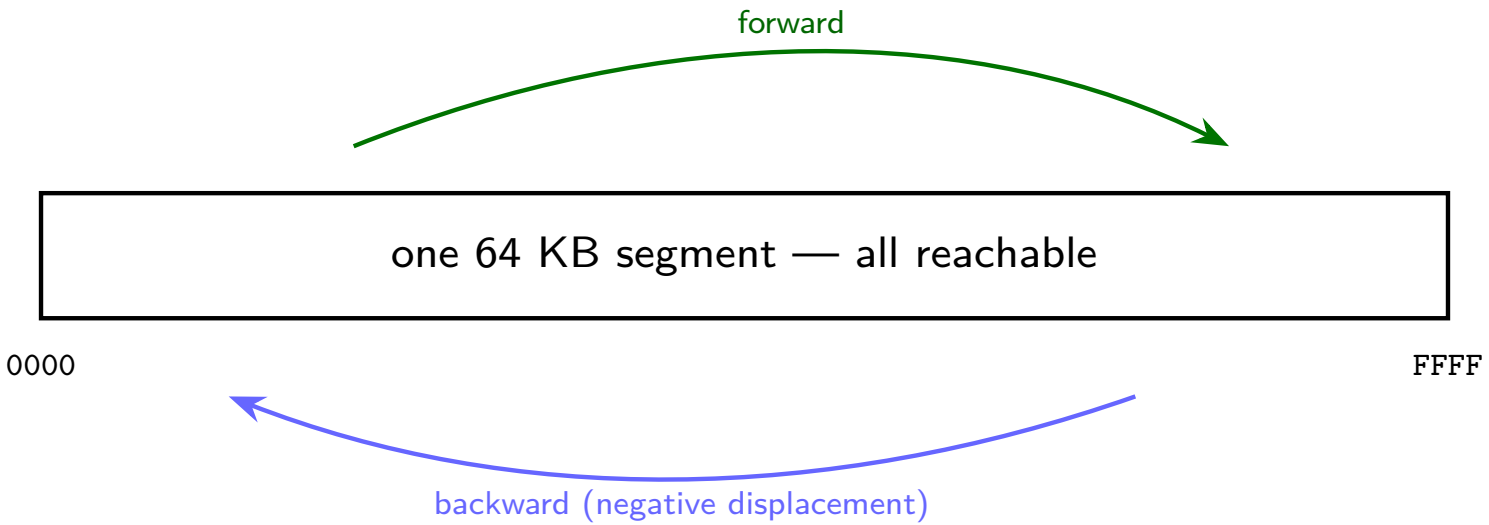


The limitation — and why

A signed byte can only hold -128 to $+127$. That is the *entire* reach of a short jump, and it comes straight from the size of one byte.

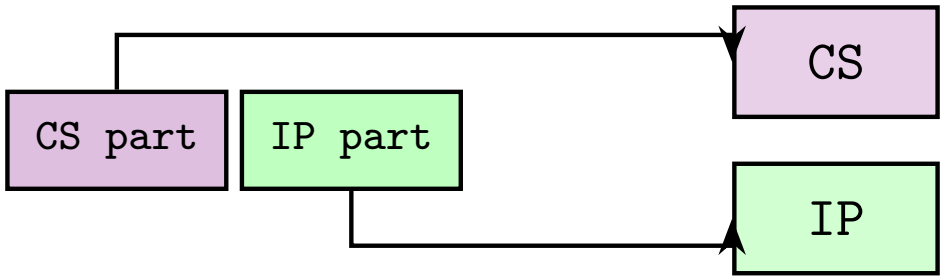
Near Jump

- The displacement is stored in **16 bits**, as in our `E9 16 00` example.
- A near jump can reach **anywhere within the segment**.
- If we add a large number, it **wraps around** to the lower part of the segment.
- A **negative** number is really just a large number, and it works precisely through this wraparound behaviour.



Far Jump

- A far jump is not position relative — it is **absolute**.
- Both a **segment** and an **offset** must be given — two bytes each, five bytes in total.
- It loads **CS** with the segment part and **IP** with the offset part, so execution resumes at that physical location.



- Short and near jumps stay **inside** one segment. Only a far jump can move from one code segment to another.
- Three instructions have a far form: `JMP`, `CALL` **and** `RET` — all program-control instructions.

A Limitation You Will Actually Hit

Instruction	Short	Near	Far
Unconditional `JMP`	✓	✓	✓
Conditional (`JE`, `JNE`, `JA`, ...)	✓	×	×

Conditional jumps can *only* be short

Every conditional jump is limited to $-128 \cdots +127$ bytes. If your loop body grows beyond that, the assembler reports *"jump out of range"*.

- **The standard workaround** — invert the condition and jump over an unconditional jump:

```
    je far_target ; if the target is too far away...
    ; becomes:
    jne skip ; invert the condition, jump a short distance
    jmp far_target ; a NEAR jump has the range
skip:
```

Why Sorting?

The first non-trivial program of the course

Until now every program did *one* thing in a straight line. Sorting needs a loop **inside** a loop, a decision in the middle, and a way to remember something across passes.

- It uses everything we have built:
  - array data (`dw`), indexed addressing (`[data+bx]`),
  - `CMP` with a conditional jump,
  - a counted loop, `JMP` to skip over data,
  - a flag variable kept **in memory**.
- It is a real algorithm — **bubble sort** — not a toy.
- And it gives the clearest possible demonstration of **signed vs unsigned**: the same data, the same algorithm, one changed jump, two completely different answers.

The Bubble Sort Idea

- Compare **two consecutive numbers**.
  - In the required order? leave them alone.
  - Out of order? **swap** them.
- Move one position forward and repeat, until the last pair has been compared.
- One complete sweep over the array is called a **pass**.
- With N elements, the simple algorithm needs $N - 1$ passes.

The improvement we will use

Keep a **swap flag**. If a whole pass completes **without a single swap**, every pair is already in order — the array is sorted, and we can stop immediately.

- **Ascending** — smallest first. **Descending** — largest first.
- The name: the largest number "bubbles" to the end of the array on the first pass, like a bubble rising through water.

Bubble Sort by Hand — Pass 1

State of Data	Swap Done	Swap Flag
Pass 1		`Off`
60 55 45 58	Yes	`On`
55 60 45 58	Yes	`On`
55 45 60 58	Yes	`On`

result of pass 1: 55 45 58 60

- Shaded = the pair currently being compared.
- **60** was swapped forward three times — it has *bubbled* to the end. After pass 1 the largest number is guaranteed to be in its final place.

Bubble Sort by Hand — Passes 2 and 3

State of Data	Swap Done	Swap Flag
Pass 2		`Off`
55 45 58 60	Yes	`On`
45 55 58 60	No	`On`
45 55 58 60	No	`On`
Pass 3		`Off`
45 55 58 60	No	`Off`
45 55 58 60	No	`Off`
45 55 58 60	No	`Off`

Pass 3 completed with **no swaps**, so the flag stayed `Off` — **no more passes are needed**.

Example 3.3 — Bubble Sort in Assembly

```
; sorting a list of ten numbers using bubble sort
[org 0x0100]
        jmp start ; jump over the data

data: dw 60, 55, 45, 50, 40, 35, 25, 30, 10, 0
swap: db 0

start: mov bx, 0 ; initialize array index to zero
        mov byte [swap], 0 ; reset swap flag to "no swaps"

loop1: mov ax, [data+bx] ; load number in ax
        cmp ax, [data+bx+2] ; compare with next number
        jbe noswap ; no swap if already in order
        mov dx, [data+bx+2] ; load second element in dx
        mov [data+bx+2], ax ; store first number in second
        mov [data+bx], dx ; store second number in first
        mov byte [swap], 1 ; flag that a swap has been done

noswap: add bx, 2 ; advance bx to next index
        cmp bx, 18 ; are we at the last index?
        jne loop1 ; if not, compare next two

        cmp byte [swap], 1 ; was any swap done in this pass?
        je start ; if yes, make another pass

        mov ax, 0x4c00 ; terminate program
        int 0x21
```

Reading the Program: the Opening Lines

- `jmp start` — placed to **skip over the data**, so that offset `0100` holds a valid instruction and the data is never executed.
- `swap: db 0` — the swap flag could have been kept in a register, but it is stored **in memory** here as an example, and to extend the concept later.
- `mov bx, 0` and `mov byte [swap], 0` — both are **inside** the `start` label, so every new pass resets the index *and* the flag.

Why `byte` is needed

`mov byte [swap], 0` — without the keyword this would be **ambiguous**: no register is involved, so the assembler could not tell whether to write one byte or two. (Week 4.)

Why Two Instructions to Compare?

```
cmp [data+bx], [data+bx+2]
```

ILLEGAL — two bracketed operands

```
mov ax, [data+bx]
cmp ax, [data+bx+2]
```

one bracket each — legal

- **Memory-to-memory comparison is not allowed** — the same rule we met in Week 4 for `mov`.
- So one element is read into `AX` first, then compared against the next element in memory.
- **A bonus:** the first element is now *already* in AX, which the swap code reuses — saving an instruction.
- Note the addressing mode: `[data+bx+2]` is **base** + **index** + **offset** — the most complex of the seven modes, used here very naturally.

The Most Important Line: JBE, Not JB

```
cmp ax, [data+bx+2]
jbe noswap ; skip the swap if already in order
```

What happens if you write JB instead

- JB does not take the jump when the two elements are **equal**, so the program **swaps equal elements**.
- Swapping equal elements sets the **swap flag** to 1.
- The flag being set forces **another pass**, which swaps them again, which sets the flag again...
- The algorithm never stops. An infinite loop.

- JBE includes the "equal" case, so equal elements are left alone and the flag stays clear.
- This is not a typing slip — it is a genuine **algorithmic flaw caused by a logic error**, and the assembler would never warn you.

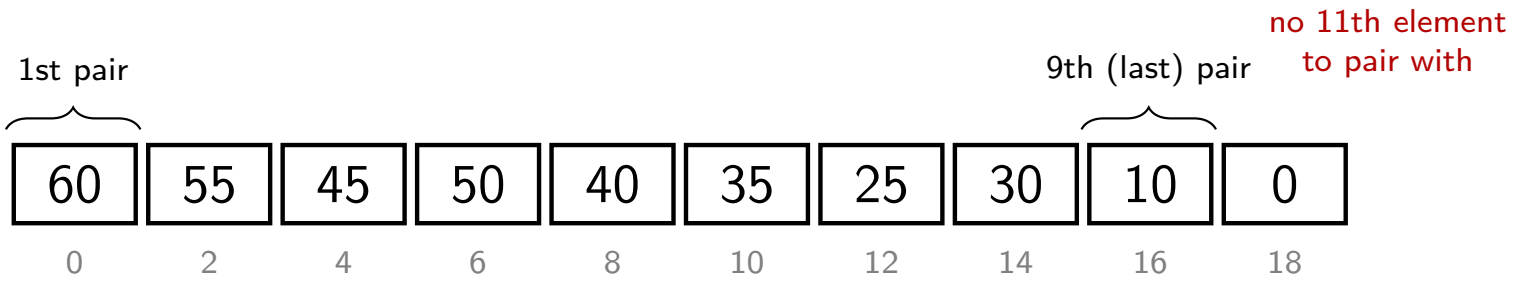
The Swap — Crossing Two Values

```
mov dx, [data+bx+2] ; second element into DX
mov [data+bx+2], ax ; AX (first) goes into the second slot
mov [data+bx], dx ; DX (second) goes into the first slot
```

After executing	[first]	[second]	AX	DX
(state after the cmp)	60	55	60	—
`mov dx, [data+bx+2]`	60	55	60	**55**
`mov [data+bx+2], ax`	60	**60**	60	55
`mov [data+bx], dx`	**55**	60	60	55
result	**55**	**60**		

- Two registers are needed because a direct exchange would **overwrite** one of the values before it is saved.
- The code reuses AX — it already holds the first element from the comparison, so it is never re-loaded.

Why `cmp bx, 18` and Not 20?

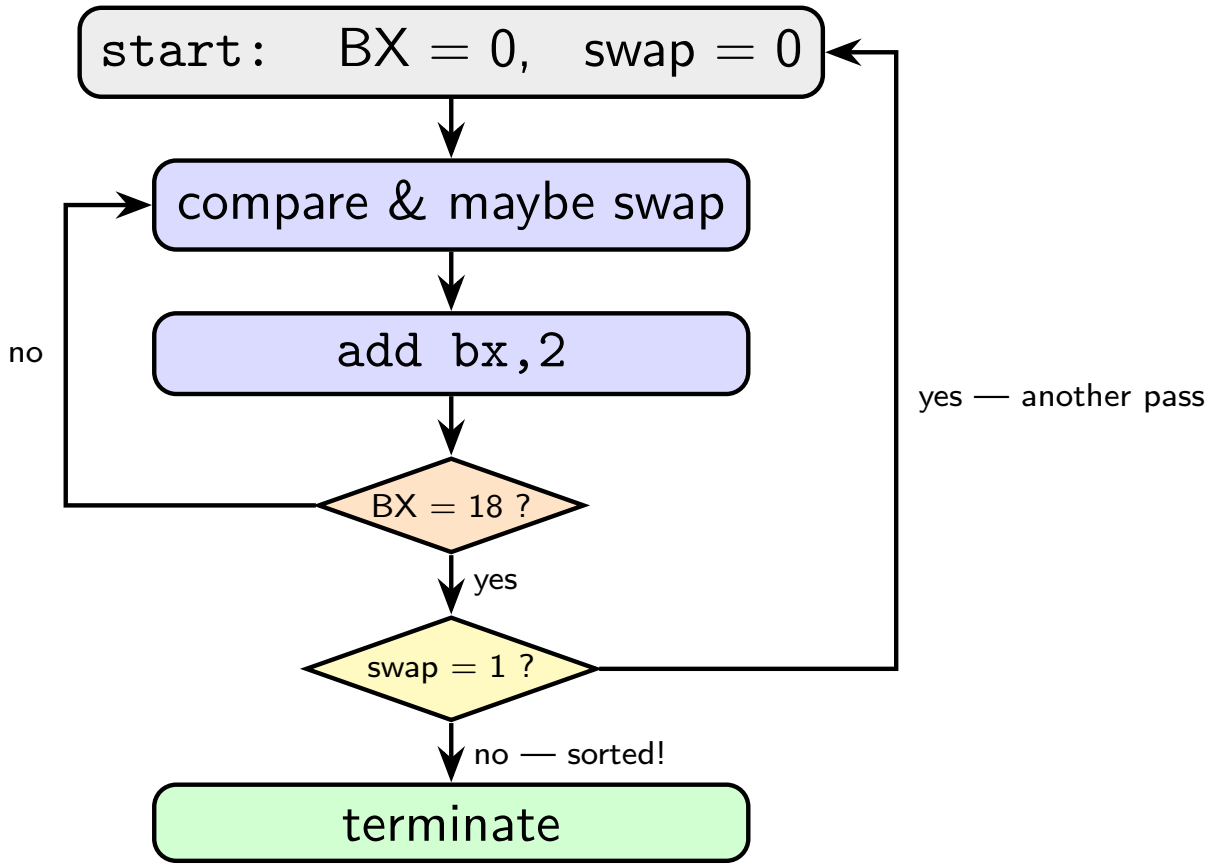


- We pick an element and compare it with the **next** one. With 10 elements there are only **9 pairs**.
- The last valid pick is the **9th** element, at index **16**. After it, BX becomes 18 and we stop.
- If we allowed BX $= 18$, we would pick the 10th element and compare it with an 11th element that does not exist — reading memory beyond our array.

Compare with Example 3.1

There we used `cmp bx, 20`, because every element was *visited*. Here every *pair* is visited — one fewer.

The Outer Loop — Repeating the Pass



- **Inner loop** walks through the array once. **Outer loop** repeats the whole pass while swaps keep happening.

Changing the Sort Order — One Instruction

	Ascending	**Descending**
Unsigned	`JBE`	`JAE`
Signed	`JLE`	`JGE`

- The **entire algorithm is unchanged**. Only the conditional jump changes — one instruction decides the whole behaviour.
- In the debugger you will see `JBE` displayed as `JNA` — the same renaming we saw with `JNE` and `JNZ`. Same opcode, two names.

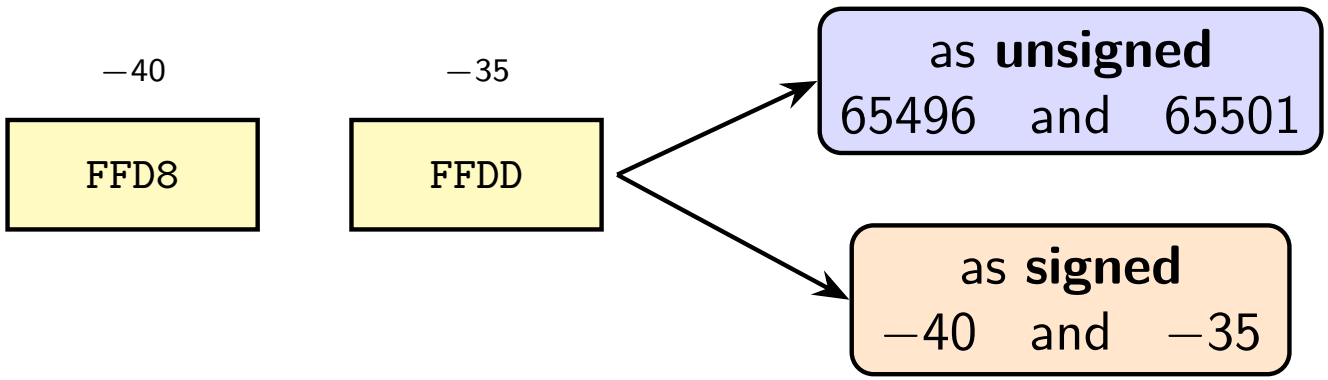
The exercise

Signed descending and unsigned descending sorts follow exactly the same pattern and are left for you to try.

The Experiment: Add Negative Numbers to the Data

```
data: dw 60, 55, 45, 50, -40, -35, 25, 30, 10, 0
```

- In the debugger the numbers appear in hexadecimal. Two of them matter most:



The data does not say whether it is signed or unsigned. Our interpretation decides.

Running the Same Program Two Ways

With JBE — unsigned

FFDD $>$ FFD8
(65501 $>$ 65496 — larger in magnitude)

Both are treated as **huge** numbers, so they finish at the **end** of the list:

... 60 FFD8 FFDD

FFDD is declared the **largest** number of all.

With JLE — signed

FFD8 $<$ FFDD
($-40 < -35$)

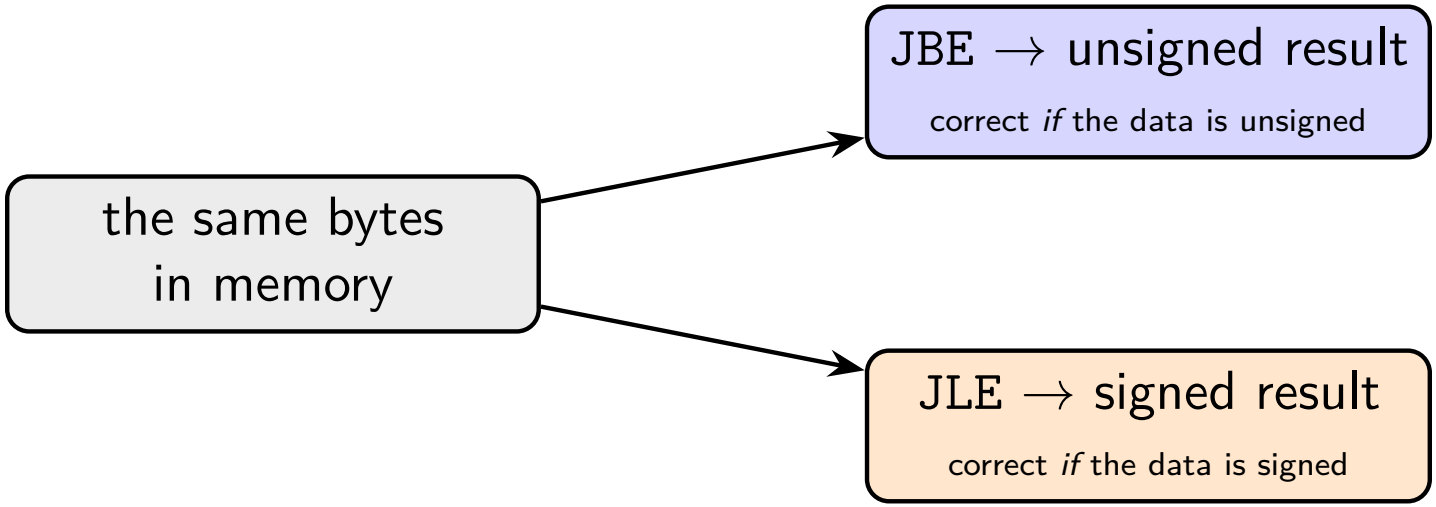
Both have the **sign bit on**, so they are smaller than **every** other number and move to the **start**:

FFD8 FFDD 0000 10 ...

FFD8 (-40) is the **smallest**.

- JBE ignores the sign bit entirely and folds it into the magnitude — so negative numbers become the *largest*.
- JLE reads the sign bit as a sign — so negative numbers become the *smallest*, below zero and below every positive.

Both Answers Are Correct



- Neither result is a bug. Each is the right answer to a *different question*.
- **This is exactly what `signed` and `unsigned` mean in C++.** There, the compiler looks at your variable's type and emits `JBE` or `JLE` for you.
- In assembly, that responsibility is yours. Choosing the jump *is* declaring the type of your data.
- It is only at this level that the actual mechanism becomes visible — in a high-level language it is hidden behind a keyword.

Summary

- Jumps store a **displacement, not an address**. The processor adds it to IP, which already points at the **next** instruction.
- **Relative addressing** lets the same code run no matter where it is loaded.
- **Short** (2 bytes, ± 128), **near** (3 bytes, whole segment), **far** (5 bytes, absolute, loads CS *and* IP).
- Conditional jumps can only be short — invert the condition and use `JMP` when the target is too far.
- **Bubble sort:** compare neighbours, swap if out of order, repeat; stop when a pass makes no swap.
- `JBE` not `JB` — swapping equal elements would loop forever.
- `cmp bx, 18` not 20 — 10 elements make only **9 pairs**.
- One instruction sets the whole behaviour: `JBE`/`JAE` for unsigned, `JLE`/`JGE` for signed; ascending or descending.

Thank You

Any Questions?

"The data never tells you what it means.
In a high-level language the compiler decides for you — in assembly, you decide, one jump at a time."

Next: the stack, procedures and `CALL`/`RET`.